

Web Storage & JSON

localStorage, sessionStorage, cookies — and using JSON to store arrays and objects as strings.

localStorage

sessionStorage

Cookies

JSON.stringify / parse

1 Browser Storage

JavaScript provides ways to store data in the browser. The main ones covered here are:

localStorage · sessionStorage · Cookies

2 localStorage

localStorage stores data in the browser that **persists even after the browser is closed**.

Data remains there until it is manually removed or cleared.

Basic operations

Method	Purpose
<code>setItem()</code>	Store/update data
<code>getItem()</code>	Retrieve data
<code>removeItem()</code>	Remove specific data
<code>clear()</code>	Remove all localStorage data

Store data

```
localStorage.setItem("name", "abrar");
```

Stores: name → abrar

Fetch data

```
let val1 = localStorage.getItem("name");
```

val1 will contain **"abrar"**

Remove specific data

```
localStorage.removeItem("name");
```

Update data

`setItem()` also updates an existing key.

```
localStorage.setItem("name", "abrar");  
  
localStorage.setItem("name", "harsh");
```

The value of `name` is now `harsh`

Clear everything

```
localStorage.clear();
```

Removes **all data** stored in localStorage for that website/origin.

3 sessionStorage

`sessionStorage` works similarly to `localStorage`, but its data is associated with the **current browser tab/session**.

It is generally removed when that page session ends, such as when the tab is closed.

Store

```
sessionStorage.setItem("name", "abrar");
```

Fetch

```
let val = sessionStorage.getItem("name");
```

Remove specific item

```
sessionStorage.removeItem("name");
```

Update

```
sessionStorage.setItem("name", "harsh");
```

If `"name"` already exists, its value is updated.

Clear everything

```
sessionStorage.clear();
```

4 localStorage vs sessionStorage



localStorage

Long-term browser storage. Persists even after the browser closes. Scoped to the origin.



sessionStorage

Temporary storage for the current tab/session. Cleared when the tab is closed.

Feature	localStorage	sessionStorage
Stores data	Yes	Yes
Data type	Strings	Strings
Persists after close	Generally yes	No
Scope	Origin	Origin + tab/session
setItem() / getItem()	Yes	Yes
removeItem() / clear()	Yes	Yes

5 Cookies

Cookies are small pieces of data stored by the browser and associated with a website.

They are commonly used for things such as:

Sessions · Login state · Preferences · Tracking

Cookies have more features and restrictions than localStorage/sessionStorage.

Creating a cookie

Your original code:

✗ INCORRECT

```
let email = document.cookie("email=harsh@gmail.com");
```

✓ CORRECT

```
document.cookie = "email=harsh@gmail.com";
```

This is incorrect because `document.cookie` is a **property**, not a function.

Reading cookies

```
console.log(document.cookie);
```

This returns the cookies available to the current document as a string.

JSON — JAVASCRIPT OBJECT NOTATION

6 JSON

JSON = JavaScript Object Notation

JSON is a text-based format commonly used for storing and transferring structured data.

A MAJOR REASON IT MATTERS HERE

`localStorage` and `sessionStorage` store values as **strings**.

7 JSON.stringify()

`JSON.stringify()` converts a JavaScript value into a JSON string.

Example

```
let friends = ["akash", "harsh", "amit"];  
  
localStorage.setItem("friends", JSON.stringify(friends));
```

The array `["akash", "harsh", "amit"]` is converted into a string before being stored.

Why?

JavaScript Array



JSON.stringify()



JSON String



localStorage

8 JSON.parse()

`JSON.parse()` converts a JSON string back into a JavaScript value.

Example:

```
let data = localStorage.getItem("friends");  
  
let friends = JSON.parse(data);
```

Now `friends` is a normal JavaScript array again: `["akash", "harsh", "amit"]`

Complete example

```
let friends = ["akash", "harsh", "amit"];
```

storage.js

```
localStorage.setItem("friends", JSON.stringify(friends));

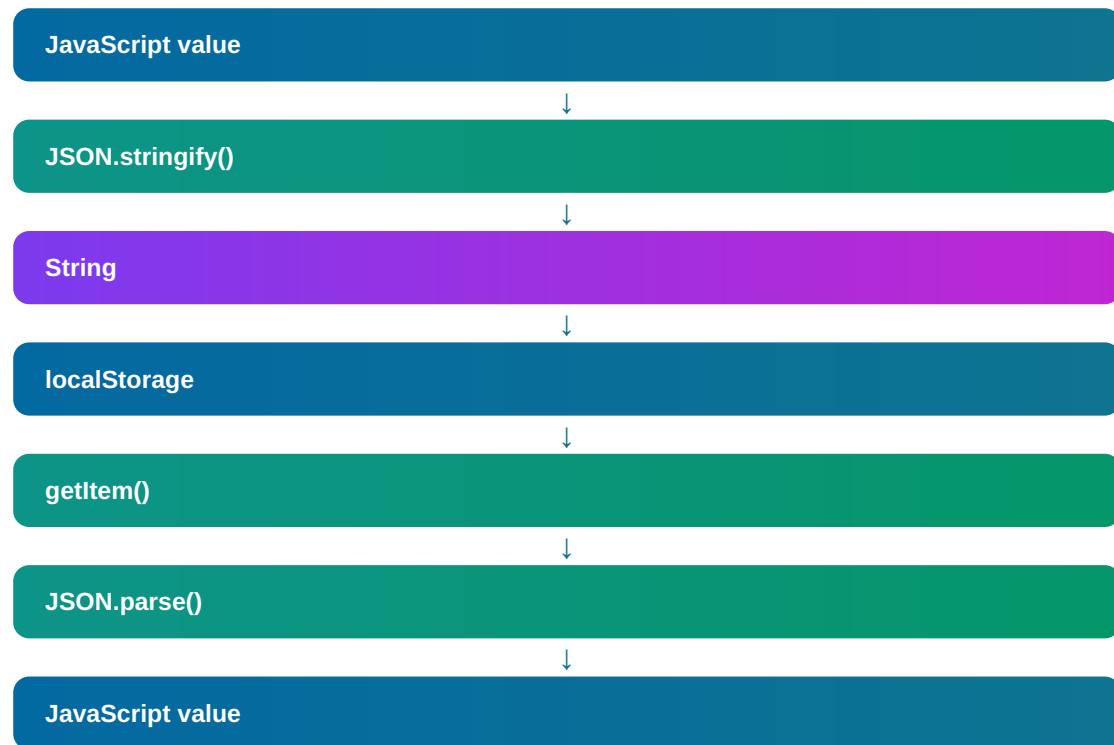
let data = localStorage.getItem("friends");

let result = JSON.parse(data);

console.log(result);
```

9 ★ Important JSON Flow

Remember this pattern:



Example

```
localStorage.setItem(
  "friends",
  JSON.stringify(["akash", "harsh", "amit"])
);

let friends = JSON.parse(
  localStorage.getItem("friends")
);
```

10 Important Correction

Your original line was:

× MISLEADING

```
let friends = localStorage.setItem(
  "friends",
  JSON.stringify(["akash", "harsh", "amit"])
);
```

This **stores the data correctly**, but assigning `setItem()` to `friends` is misleading because `setItem()` does not return the stored array.

Better:

✓ STEP 1 — STORE

```
localStorage.setItem(
  "friends",
  JSON.stringify(["akash", "harsh", "amit"])
);
```

✓ STEP 2 — RETRIEVE

```
let friends = JSON.parse(
  localStorage.getItem("friends")
);
```

Now `friends` actually contains the JavaScript array.

11 Quick Revision

localStorage

```
localStorage.setItem("key", "value");
localStorage.getItem("key");
localStorage.removeItem("key");
localStorage.clear();
```

sessionStorage

```
sessionStorage.setItem("key", "value");
sessionStorage.getItem("key");
sessionStorage.removeItem("key");
sessionStorage.clear();
```

Cookies

```
document.cookie = "key=value";
console.log(document.cookie);
```

JSON

```
JSON.stringify(value); // JS -> JSON string

JSON.parse(value);     // JSON string -> JS
```

Most important concept



🎯 FINAL THINGS TO REMEMBER

- `localStorage` → persists after closing the browser.
- `sessionStorage` → temporary, tied to the current tab/session.
- Both storage APIs use **string values**.
- `setItem()` can both **create and update** a key.
- `removeItem()` removes one key.
- `clear()` removes all storage data for that storage area.
- `document.cookie` is a **property**, not a function.
- `JSON.stringify()` → JavaScript value → JSON string.
- `JSON.parse()` → JSON string → JavaScript value.

✨ Storage APIs only speak strings — `JSON.stringify()` and `JSON.parse()` are the bridge to real JS data.